

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 05

Prediction Before the Outcome Is Known

Use real AAPL prices to build a chronological prediction table. Begin with a simple linear model, check its baselines, then change the training window without changing what the target means. Week 4 supplies the execution rule; costs and full performance evaluation follow in Weeks 6-7.

Issue	Question	Page
1	Which return is the model predicting?	3
2	Which targets are known when a prediction is made?	4
3	Where do training, validation and test periods end?	5
4	Why can random splitting use the future?	6
5	What does least squares fit?	7
6	How does one feature become one prediction?	8
7	Which observations may determine a scaling rule?	9
8	Why compare with zero and the historical mean?	10
9	Which prediction errors are being averaged?	11
10	How do expanding and rolling samples differ?	12
11	How is a walk-forward prediction made?	13
12	Can predictions change without refitting?	14
13	When may we choose the window length?	15
14	Is the predicted return the return earned after entry?	16
15	Does lower prediction error imply more money?	17
16	Can the final feature row be predicted but not scored?	18

Use the examples in order. Each issue ends with two discussion questions, not a separate submission. Market observations and teaching assumptions are identified separately.

SHARED DATA AND SETTINGS

Prepare the common example

We extend the price history into 2023 to supply earlier training observations. Evaluation still ends June 28, 2024. Close is the provider price field, not a dividend-reinvested return or a verified fill. All dates count stored observations. The assumed daily prices are available after their Close and before the next assumed execution.

Before running: Which first and last observations define the downloaded history?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
aapl = yf.Ticker("AAPL").history(start="2023-01-01",
                                  end="2024-07-01", auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
close = aapl["Close"]
assert close.index.is_unique and close.index.is_monotonic_increasing
assert np.isfinite(close).all() and (close > 0).all()
pd.DataFrame({"Date": close.index[[0,-1]],
              "Close (USD/share)": close.iloc[[0,-1]].round(2).values})
```

Date	Close (USD/share)
2023-01-03	125.07
2024-06-28	210.62

The response contains 374 observations. Keep this same response for every comparison below. The additional 2023 history changes model training, not the 2024 evaluation boundaries.

The examples are retrospective teaching comparisons. Their dates and model choices are not a preregistered investment test or the official Report 1 dataset. No actual orders are placed.

ISSUE 01

Which return is the model predicting?

A prediction needs a specific outcome date. Here we predict the next stored close-to-close return, not a future price level.

C is Close in USD per share. The subscript t names a stored trading date; $t+1$ is the next observation, not necessarily the next calendar day. The target y is a decimal price return. Dividends and costs are excluded.

$$y_t = \frac{C_{t+1}}{C_t} - 1$$

$$y_{\text{Jan 2}} \approx \frac{184.25000}{185.64000} - 1 \approx -0.007488$$

Before running: Will the January 2 target be negative? Which date supplies its ending price?

```
dates = pd.Series(close.index, index=close.index)
model = pd.DataFrame({"x": close / close.shift(5) - 1,
                     "y": close.shift(-1) / close - 1,
                     "End": dates.shift(-1)})
show = model.loc["2024-01-02":"2024-01-03"]
pd.DataFrame({"Decision": show.index, "Outcome": show["End"],
              "Return (%)": (100 * show["y"]).round(3)})
```

Decision	Outcome	Return (%)
2024-01-02	2024-01-03	-0.749
2024-01-03	2024-01-04	-1.27

The January 2 target is -0.749%, using the January 3 Close. It is not known after the January 2 Close. The feature x is the five-interval price return introduced in Week 3; its calculation uses only prices through t .

Discuss

1. Would changing the target to the January 4 Close leave the financial question unchanged? Specify the new holding interval.
2. A file labels this target January 2. May its value enter a model fitted that evening? Identify the missing price.

Sources: [1], [2], [7]. See the references page.

ISSUE 02

Which targets are known when a prediction is made?

A row may enter training only after both its feature and its outcome are available. A historical download contains later answers, so the program must enforce the date restriction.

Assume prices become available just after Close. At that moment, a target ending today is available; a target ending at the next Close is not. This assumes no publication or processing delay.

Before running: Is the December 29 row eligible on January 2? Is the January 2 row eligible?

```
decision = "2024-01-02"
known = model.dropna()
eligible = known.loc[known["End"] <= decision]
recent = model.loc["2023-12-28":"2024-01-02"]
pd.DataFrame({"Feature date": recent.index,
              "Outcome date": recent["End"],
              "Known now?": recent["End"].le(decision).map({True:"Yes", False:"No"})})
```

Feature date	Outcome date	Known now?
2023-12-28	2023-12-29	Yes
2023-12-29	2024-01-02	Yes
2024-01-02	2024-01-03	No

December 29 is eligible because its target ends January 2. The January 2 row is not eligible because its target ends January 3. Dropping only the current feature row is insufficient for targets that end several observations later.

The rule concerns information availability, not the row number. Keep the feature date and outcome date when checking a model table. A row can be useful for prediction before it becomes useful for training.

Discuss

1. If fitting happens before the January 2 Close, which displayed Yes must become No? Explain using the ending price.
2. If the target ends five observations later, is excluding only today enough? Give a date-based training condition.

Sources: [4], [5]. See the references page.

ISSUE 03

Where do training, validation and test periods end?

Training estimates coefficients. Validation compares choices. A later test period evaluates a choice that has already been made. None of these names makes a period independent of our decisions.

We fit the first fixed model using outcomes through September 29, 2023. October-December is reserved for validation; January-June 2024 is the later evaluation period. Boundary rows whose outcomes cross a cutoff are omitted.

Before running: Can a September 29 feature with an October 2 outcome enter the training period?

```
train = known.loc[known["End"] <= "2023-09-29"]
valid = known.loc[(known.index >= "2023-10-02") &
                  (known["End"] <= "2023-12-29")]
test = known.loc[known.index >= "2024-01-02"]
parts = {"Train": train, "Validate": valid, "Test": test}
pd.DataFrame([{"Period": name, "First feature": frame.index[0],
              "Last outcome": frame["End"].iloc[-1], "Rows": len(frame)}
             for name, frame in parts.items()])
```

Period	First feature	Last outcome	Rows
Train	2023-01-10	2023-09-29	181
Validate	2023-10-02	2023-12-29	62
Test	2024-01-02	2024-06-28	123

There are 181 training rows, 62 validation rows and 123 test rows. The omitted cutoff rows are intentional, not missing market data. The final test target ends June 28.

This is a retrospective classroom example. Because these dates have already been inspected, the lesson is not evidence of a genuinely untouched live test. Repeatedly changing the model after reading its test result turns that period into further development data.

Discuss

1. A model is changed after its June result is disappointing. Can the same January-June result remain a final test? State what evidence is now needed.
2. Why should the last outcome date, rather than only the last feature date, determine the training cutoff? Use September 29 and October 2.

Sources: [2], [4], [5]. See the references page.

ISSUE 04

Why can random splitting use the future?

Random splitting can assign later observations to training and earlier observations to evaluation. That does not reproduce a model that had to make decisions in date order.

Before running: For the earliest randomly held-out date, will any training outcomes come from later dates?

```
random_train = known.sample(frac=0.7, random_state=7)
random_test = known.drop(random_train.index).sort_index()
early = random_test.index[0]
future = random_train.loc[random_train["End"] > early]
pd.DataFrame({"Evaluation date": [early],
             "Later training outcomes": [len(future)],
             "Last training outcome": [random_train["End"].max()]})
```

Evaluation date	Later training outcomes	Last training outcome
2023-01-10	258	2024-06-28

The held-out date 2023-01-10 has 258 later outcomes inside the random training sample. Calling that row test data does not make the model historically available on that date.

Random splitting answers a different sampling question. For this chronological trading decision, use earlier eligible observations to predict later outcomes. Chronological splitting still does not remove dependence, changes in market conditions or repeated model selection.

Discuss

1. Would sorting the random training rows after sampling fix this experiment? Explain whether it removes later outcomes.
2. A chronological split uses a scaler fitted on the entire dataset. Is the evaluation now valid? Identify the remaining future information.

Sources: [4], [5]. See the references page.

ISSUE 05

What does least squares fit?

Ordinary least squares chooses a line that minimizes the sum of squared training errors. It does not maximize trading profit.

x is the five-interval price return; y is the next-interval price return, both decimal. The fitted intercept b_0 and slope b_1 give prediction $\hat{y}$. Bars denote training means; sums run over training rows. Nonconstant x is required.

$$\hat{y}_i = b_0 + b_1 x_i, \quad \min_{b_0, b_1} \sum_i (y_i - b_0 - b_1 x_i)^2$$

The intercept derivative gives $b_0 = \text{mean } y - b_1 \times \text{mean } x$. Substituting this into the slope equation gives:

$$b_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$b_1 \approx \frac{-0.0027354}{0.1714426} \approx -0.015955$$

Before running: Will the slope have the same sign as the displayed numerator?

```
X = np.column_stack([np.ones(len(train)), train["x"]])
b0, b1 = np.linalg.lstsq(X, train["y"], rcond=None)[0]
pd.DataFrame({"Fit ends": [train["End"].max()],
              "Intercept": [round(b0, 6)], "Slope": [round(b1, 6)]})
```

Fit ends	Intercept	Slope
2023-09-29	0.001722	-0.015955

The slope is -0.015955. It describes this fitted association, not a causal response to trading. The intercept is 0.001722 decimal return; it is not an account balance.

Discuss

1. If every training x were identical, which denominator would fail? Could the data identify a unique slope?
2. Does a positive fitted intercept establish profitable future trades? Name the outcome-date and execution checks still needed.

Sources: [2], [3]. See the references page.

ISSUE 06

How does one feature become one prediction?

Once the coefficients are fixed, a new feature value produces a new prediction. Fitting and predicting are different operations.

C is Close in USD/share. The January 2 feature x compares December 22 with January 2, five stored price intervals apart. The prediction is a decimal return for January 2 to January 3; b_0 and b_1 are the fixed fitted intercept and slope.

$$x_t = \frac{C_t}{C_{t-5}} - 1, \quad \hat{y}_t = b_0 + b_1 x_t$$

$$x_t \approx \frac{185.64000}{193.60001} - 1 \approx -0.041116$$

$$\hat{y}_t \approx 0.001722 + (-0.015955)(-0.041116) \approx 0.002378$$

Before running: Does the January 2 prediction have the same sign as its actual next-close return?

```
fixed = b0 + b1 * test["x"]
preview = test.iloc[:3]
pd.DataFrame({"Date": preview.index,
              "Feature (%)": (100 * preview["x"]).round(3),
              "Predicted (%)": (100 * fixed.iloc[:3]).round(3),
              "Actual (%)": (100 * preview["y"]).round(3)})
```

Date	Feature (%)	Predicted (%)	Actual (%)
2024-01-02	-4.112	0.238	-0.749
2024-01-03	-4.558	0.245	-1.27
2024-01-04	-5.819	0.265	-0.401

The prediction is 0.238%; the actual outcome is -0.749%. The model predicts an average relationship, not an exact future price. Display rounding does not change calculations.

Discuss

1. A student replaces the feature with a return ending January 3. Why would the January 2 prediction no longer be available on time?
2. If x changes but b_0 and b_1 do not, has the model been refitted? Identify which operation changed.

Sources: [1], [2], [3]. See the references page.

ISSUE 07

Which observations may determine a scaling rule?

A scaling rule is estimated from data. If it uses future observations, the transformed historical inputs contain information unavailable at the decision date.

x is the five-interval decimal price return; n counts training rows. Let μ be its training mean and s its training standard deviation. Here scaling uses divisor n , unlike Week 3's sample divisor $n-1$. The standardized feature z has no units. Freeze μ and s for later rows; do not fit them on the test period.

$$z_t = \frac{x_t - \mu}{s}, \quad s > 0$$

$$z_{\text{Jan 2}} \approx \frac{-0.041116 - (0.008966)}{0.030777} \approx -1.6273$$

Before running: Will a feature below the training mean have a negative standardized value?

```
mu, scale = train["x"].mean(), train["x"].std(ddof=0)
z = (test["x"] - mu) / scale
pd.DataFrame({"Date": test.index[:3],
              "Feature (%)": (100 * test["x"].iloc[:3]).round(3),
              "Standardized": z.iloc[:3].round(4)})
```

Date	Feature (%)	Standardized
2024-01-02	-4.112	-1.6273
2024-01-03	-4.558	-1.7724
2024-01-04	-5.819	-2.1821

The January 2 standardized feature is -1.6273. This is a distance measured in training standard deviations, not a percentage return.

Our unpenalized one-feature regression with an intercept does not require scaling: correctly refitting after this invertible transformation gives the same predictions. Scaling can matter for penalized or distance-based methods, but it never permits using future fitting data.

Discuss

1. Can tomorrow's feature be standardized without knowing tomorrow's target? State which fixed quantities are needed.
2. A full-history scaler changes the coefficients but not this OLS model's predictions. Does that excuse the general preprocessing procedure? Explain the historical availability requirement.

Sources: [2], [5]. See the references page.

ISSUE 08

Why compare with zero and the historical mean?

A fitted model should be compared with predictions that require little or no modeling. We use zero return and the mean of the same training targets.

The historical-mean prediction is constant across later dates. Here n is the number of training targets; the sum and mean use only those targets, in decimal returns. Zero predicts no price change; it is not a claim that markets have zero expected return.

$$\hat{y}_t^{\text{mean}} = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad \hat{y}_t^{\text{zero}} = 0$$

$$\bar{y} \approx \frac{0.285821}{181} \approx 0.001579$$

Before running: Which predictions can change from one date to the next without fitting again?

```
predictions = pd.DataFrame({"OLS": fixed,
                           "Zero": 0.0, "Mean": train["y"].mean()})
preview = (100 * predictions.iloc[:3]).round(3)
preview.columns = [name + " (%)" for name in preview.columns]
preview.insert(0, "Date", preview.index)
preview
```

Date	OLS (%)	Zero (%)	Mean (%)
2024-01-02	0.238	0.0	0.158
2024-01-03	0.245	0.0	0.158
2024-01-04	0.265	0.0	0.158

All prediction columns are percentages. The historical mean stays at 0.158%; OLS changes with x . Every method predicts the same dated target. None includes costs or a portfolio-weight choice.

Discuss

1. If the mean uses January-June outcomes, is it still an eligible September-fitted baseline? Identify the information being added.
2. A model beats zero but not the historical mean. What narrower conclusion is supported? Name both comparisons.

Sources: [2], [4]. See the references page.

ISSUE 09

Which prediction errors are being averaged?

Mean absolute error measures typical absolute prediction error. Mean squared error gives larger errors more influence. Both require identical target dates across models.

Define e as actual minus predicted return in percentage points (pp). MAE is measured in pp; MSE is measured in pp^2 . Here n is the number of evaluated targets, not the number of prices.

$$e_t = 100(y_t - \hat{y}_t), \quad \text{MAE} = \frac{1}{n} \sum_t |e_t|, \quad \text{MSE} = \frac{1}{n} \sum_t e_t^2$$

$$e_{\text{Jan 2}} \approx -0.7488 - (0.2378) \approx -0.9866 \text{ pp}$$

Before running: Which criterion gives a few unusually large errors more weight?

```
errors = predictions.rsub(test["y"], axis=0) * 100
scores = pd.DataFrame({"Model": errors.columns,
    "MAE (pp)": errors.abs().mean().values,
    "MSE (pp²)": errors.pow(2).mean().values})
scores.round(4)
```

Model	MAE (pp)	MSE (pp ²)
OLS	1.0949	2.3049
Zero	1.0921	2.323
Mean	1.0962	2.3119

OLS has MAE 1.0949 pp and MSE 2.3049 pp^2 over 123 targets. These are prediction losses, not a loss of that many dollars. The January 2 squared contribution is 0.9733 pp^2 before averaging.

Discuss

1. If one method omits its five largest errors, can its displayed MSE be compared with these rows? State the common-date correction.
2. A forecast has smaller absolute error but the wrong sign. Must it produce the preferred trade? Explain what prediction loss does not specify.

Sources: [2], [6]. See the references page.

ISSUE 10

How do expanding and rolling samples differ?

An expanding training sample keeps all eligible history. A rolling sample keeps only the most recent specified number of eligible rows. Both must respect outcome availability.

We use 80 rows to illustrate a rolling window, not because it is known to be optimal. The window length counts completed training targets, not calendar days. Older observations add information but may describe different market conditions.

Before running: Which sample grows between January and June? Which retains 80 rows?

```
rows = []
for day in ["2024-01-02", "2024-06-27"]:
    history = known.loc[known["End"] <= day]
    for name, sample in [("Expanding", history),
                        ("Rolling 80", history.tail(80))]:
        rows.append([day, name, len(sample), sample.index[0]])
pd.DataFrame(rows, columns=["Decision", "Sample", "Rows", "First feature"])
```

Decision	Sample	Rows	First feature
2024-01-02	Expanding	245	2023-01-10
2024-01-02	Rolling 80	80	2023-09-07
2024-06-27	Expanding	367	2023-01-10
2024-06-27	Rolling 80	80	2024-03-04

The expanding sample grows from 245 to 367 rows. The rolling sample remains at 80 and moves its first feature date from 2023-09-07 to 2024-03-04. Neither rule proves that its future error will be smaller.

Discuss

1. For June 27, which observations does rolling discard that expanding retains? Explain a possible benefit and a possible cost without claiming either occurs here.
2. If only 50 eligible rows exist, can the implementation truthfully report a full 80-row fit? Specify a minimum-history decision.

Sources: [4], [5]. See the references page.

ISSUE 11

How is a walk-forward prediction made?

Walk-forward evaluation repeats a historical decision: collect known outcomes, fit the model, then predict a later outcome. A later fit must not rewrite an earlier prediction.

Here the model is refitted after each Close using the latest 80 eligible rows. The target and feature remain unchanged. Updating on an outcome after it becomes known is allowed by this rule; selecting a new rule after inspecting the test scores is different.

Before running: Should any last training outcome date be later than its prediction decision date?

```
walk = pd.Series(index=test.index, dtype=float)
last_outcome = []
for day in test.index:
    history = known.loc[known["End"] <= day].tail(80)
    assert len(history) == 80
    X = np.column_stack([np.ones(80), history["x"]])
    coef = np.linalg.lstsq(X, history["y"], rcond=None)[0]
    walk.loc[day] = coef[0] + coef[1] * test.loc[day, "x"]
    last_outcome.append(history["End"].iloc[-1])
pd.DataFrame({"Decision": test.index[:3],
             "Last known outcome": last_outcome[:3],
             "Predicted (%)": (100 * walk.iloc[:3]).round(3)})
```

Decision	Last known outcome	Predicted (%)
2024-01-02	2024-01-02	-0.227
2024-01-03	2024-01-03	-0.315
2024-01-04	2024-01-04	-0.53

January 2 predicts -0.227% using outcomes known by January 2. The next row uses a later eligible sample. The code stores each prediction at its own decision date; it never replaces it using the final fit.

Discuss

1. What changes if all predictions are recalculated with the June coefficients? Identify which historical decisions become impossible.
2. Why does using January 2's completed outcome for a January 2 evening fit not mean using the January 3 outcome? Name the feature date of the completed row.

Sources: [4], [5]. See the references page.

ISSUE 12

Can predictions change without refitting?

Refitting changes coefficients. Recalculating a prediction uses a new feature with the current coefficients. A trading rule could act on either schedule, but they are not the same decision.

Keep the 80-row training window and daily feature calculation. Change only the refitting schedule: every fifth prediction date instead of every date. No trading costs or new position rule are introduced here.

Before running: Can the first three predictions differ even though their fit date is identical?

```
less_often = pd.Series(index=test.index, dtype=float)
fit_dates = []
for j, day in enumerate(test.index):
    if j % 5 == 0:
        history = known.loc[known["End"] <= day].tail(80)
        X = np.column_stack([np.ones(80), history["x"]])
        coef = np.linalg.lstsq(X, history["y"], rcond=None)[0]
        fitted_on = day
        less_often.loc[day] = coef[0] + coef[1] * test.loc[day, "x"]
        fit_dates.append(fitted_on)
pd.DataFrame({"Decision": test.index[:3], "Fitted on": fit_dates[:3],
             "Predicted (%)": (100 * less_often.iloc[:3]).round(3)})
```

Decision	Fitted on	Predicted (%)
2024-01-02	2024-01-02	-0.227
2024-01-03	2024-01-02	-0.257
2024-01-04	2024-01-02	-0.341

The first two predictions are -0.227% and -0.257%. Both use the January 2 coefficients. Changing x explains why a prediction can change without refitting.

Discuss

1. Would trading only when the model is refitted be the same experiment? Identify the additional rule that would change.
2. If the five-date schedule and daily schedule have different errors, what can be attributed to refitting frequency here? List the settings held fixed.

Sources: [2], [4]. See the references page.

ISSUE 13

When may we choose the window length?

Choose a window using validation outcomes, then fix the choice before evaluating a later period. Looking at the later scores and choosing again consumes that evidence for development.

Compare 60 and 120 eligible rows on exactly the same October-December validation targets. Both use the same feature, daily refitting and MSE in percentage-points squared. These are illustrative choices, not an exhaustive search.

Before running: If 60 rows has smaller validation MSE, which choice should be fixed before the later test?

```
validation_mse = {}
for length in [60, 120]:
    forecast = []
    for day in valid.index:
        h = known.loc[known["End"] <= day].tail(length)
        assert len(h) == length
        X = np.column_stack([np.ones(length), h["x"]])
        coef = np.linalg.lstsq(X, h["y"], rcond=None)[0]
        forecast.append(coef[0] + coef[1] * valid.loc[day, "x"])
    e = 100 * (valid["y"].to_numpy() - np.array(forecast))
    validation_mse[length] = np.mean(e ** 2)
pd.DataFrame({"Window (rows)": list(validation_mse),
              "Validation MSE (pp²)": np.round(list(validation_mse.values()), 4)})
```

Window (rows)	Validation MSE (pp²)
60	0.9449
120	0.9498

The displayed validation comparison selects 60 rows: MSE 0.9449 pp². This is a selection result, not evidence that this window is universally preferable or will win in January-June.

Discuss

1. After seeing the test results, a student switches to the other window. What role does that test period now play?
2. If one window were scored only on low-volatility dates, could this table isolate window length? State the required common-date check.

Sources: [2], [4], [5], [6]. See the references page.

ISSUE 14

Is the predicted return the return earned after entry?

The model predicts the return from today's Close to the next Close. Week 4 instead assumes the order executes at the next Close. That holding earns its first return one observation later.

The forecast made after January 2 targets January 2-3. Entry at January 3 Close earns January 3-4 if held. Here C is Close in USD/share, t is the decision date, and y and r are decimal returns. These intervals differ even when both are called one day.

$$y_t = \frac{C_{t+1}}{C_t} - 1, \quad r_{t+2} = \frac{C_{t+2}}{C_{t+1}} - 1$$

$$y_t \approx \frac{184.25000}{185.64000} - 1 \approx -0.007488, \quad r_{t+2} \approx \frac{181.91000}{184.25000} - 1 \approx -0.012700$$

Before running: For the January 2 signal, is the first earned return the January 3 or January 4 ending return?

```
held_return = close.shift(-2) / close.shift(-1) - 1
pd.DataFrame({"Decision": test.index[:3],
              "Entry": dates.shift(-1).loc[test.index[:3]],
              "First return ends": dates.shift(-2).loc[test.index[:3]],
              "Earned (%)": (100 * held_return.loc[test.index[:3]]).round(3)})
```

Decision	Entry	First return ends	Earned (%)
2024-01-02	2024-01-03	2024-01-04	-1.27
2024-01-03	2024-01-04	2024-01-05	-0.401
2024-01-04	2024-01-05	2024-01-08	2.417

The target is -0.749%, whereas the January 3-4 price return is -1.270%. A positive forecast is not a fill at the already observed January 2 Close.

Discuss

1. If the intended goal is to predict the first earned interval, how must the target and its availability date change?
2. Why is shifting a signal by one row insufficient for this stated next-Close execution rule? Trace decision, execution and first earned return.

Sources: [1], [7]. See the references page.

ISSUE 15

Does lower prediction error imply more money?

Prediction error evaluates the size of forecast mistakes. A trading rule uses forecasts to choose exposure. The two calculations answer different questions.

Keep each forecast rule fixed. A positive prediction produces stock exposure 1; otherwise exposure is 0. Under next-Close execution, p at date t uses the signal from $t-2$. Start with USD 1,000 in cash; allow fractional shares, zero cash interest and no costs or dividends.

$$s_t = \mathbf{1}(\hat{y}_t > 0), \quad p_t = s_{t-2}, \quad W_t = W_{t-1}(1 + p_t r_t)$$

W is wealth in USD; r is the decimal price return ending at t . Mean predicts positive on January 2, enters January 3, then earns the January 3-4 return:

$$W_{\text{Jan 4}} \approx 1000(1 + 1 \times (-0.012700)) \approx 987.30 \text{ USD}$$

Before running: Do Zero and Mean necessarily produce the same positions because both predictions are constant?

```
prices_2024 = close.loc["2024-01-02":]
r = prices_2024.pct_change(fill_method=None).fillna(0)
wealth = {}
for name in predictions:
    signal = predictions[name].reindex(r.index).gt(0).astype(float)
    position = signal.shift(2).fillna(0)
    wealth[name] = 1000 * (1 + position * r).cumprod()
account = pd.DataFrame(wealth)
pd.DataFrame({"Model": scores["Model"],
              "MSE (pp²)": scores["MSE (pp²)"].round(4),
              "Final gross (USD)": account.iloc[-1].values.round(2)})
```

Model	MSE (pp²)	Final gross (USD)
OLS	2.3049	1168.25
Zero	2.323	1000.0
Mean	2.3119	1143.12

Zero ends at USD 1000.00; Mean ends at USD 1143.12; OLS ends at USD 1168.25. These are gross, price-only outcomes under one assumed execution rule, not a complete strategy ranking.

Discuss

1. Could two predictions have different MSE but identical signs on every date? Explain the account consequence under this exact rule.
2. Can these balances establish which strategy is preferable after trading costs? Identify the missing evidence without changing the prediction target.

Sources: [2], [6], [7]. See the references page.

ISSUE 16

Can the final feature row be predicted but not scored?

The last complete feature row can support a prediction even when its future target is missing. Missing outcomes must not be replaced by zero for scoring.

The snapshot ends June 28. Its five-interval feature is available, but its next-close target needs a later price outside this snapshot. Prediction and scoring therefore have different row requirements.

Before running: Will the June 28 row have a prediction, an observed target, both or neither?

```
latest = model.iloc[-2:].copy()
latest["Prediction (%)"] = 100 * (b0 + b1 * latest["x"])
latest["Actual (%)"] = (100 * latest["y"]).round(3)
latest["Scorable?"] = latest["y"].notna()
pd.DataFrame({"Date": latest.index,
              "Prediction (%)": latest["Prediction (%)"].round(3),
              "Actual (%)": latest["Actual (%)"].fillna("Unavailable"),
              "Scorable?": latest["Scorable?"]})
```

Date	Prediction (%)	Actual (%)	Scorable?
2024-06-27	0.139	-1.625	True
2024-06-28	0.148	Unavailable	False

June 28 has prediction 0.148% but no target in the snapshot. June 27 can be scored because June 28 supplies its ending price. A missing future outcome is not evidence of a flat market.

Week 6 now fixes a strategy and execution schedule, then adds stated cost assumptions. Week 7 evaluates its returns and uncertainty. Week 8 develops equal-weighted holdings and rebalancing; this prediction-loss comparison does not select portfolio weights.

Discuss

1. Why is dropping all rows with missing targets appropriate for retrospective scoring but inappropriate for preparing the next prediction?
2. If a later snapshot supplies the next Close, what must stay unchanged before scoring the saved June 28 prediction?

Sources: [1], [4], [5]. See the references page.

REFERENCES AND REPRODUCTION

Sources and limits

[1] [yfinance](#): historical price fields

[2] James et al.: [An Introduction to Statistical Learning](#), Chapters 2-3

[3] [NumPy](#): least-squares computation

[4] [scikit-learn](#): chronological evaluation

[5] [scikit-learn](#): preprocessing and data leakage

[6] [scikit-learn](#): mean squared error

[7] [pandas](#): shifting observations

The least-squares identities and losses describe prediction, not market causality. All account calculations are gross and price-only under Week 4 assumptions. Short windows illustrate updating and are not recommended trading settings. Older notebook examples are not required to run this lesson.

Executed with Python 3.12.9, NumPy 2.2.6, pandas 2.2.3, matplotlib 3.10.8, yfinance 1.2.0. Run displayed code in order in a fresh Python session. Data requests need internet access; historical provider responses may change.

Selected cases illustrate the stated calculations, not investment recommendations or future-performance guarantees. Full snapshots and instructor checks are not included in the public PDF.